

DOKUMEN JAWABAN TES SELEKSI TECHNICAL ASSESSMENT

BACKEND ENGINEER POSITION — REQUIREMENTS & TEST CASES

RINGKASAN EKSEKUTIF (EXECUTIVE SUMMARY)

Dokumen ini merupakan jawaban komprehensif atas seluruh persyaratan teknis dan studi kasus yang tercantum pada berkas **Backend Engineer Position Requirements & Test Cases.pdf**. Seluruh implementasi solusi telah dibuat, diuji, dan diorganisir secara rapi di dalam repositori codebase ini ke dalam 4 modul utama:

Modul / Case	Teknologi Utama	Fokus & Arsitektur	Status Implementasi
Case 1: laravel-app	Laravel 13, PHP 8.3, JWT Auth, SQLite/MySQL	RESTful API Omzet Merchant & Outlet, TenantScope Multi-tenancy, Swagger OpenAPI 3.0, Composite B-Tree Indexing DML.	100% Selesai & Pass Test
Case 2: go_rest_api	Go 1.21+, Fiber, PostgreSQL, Docker, gRPC, NATS	Clean Architecture (Uncle Bob), Auth JWT, Task Management State Machine (todo→in_progress→done), Translation Service & History audit log.	100% Selesai & Pass Test
Case 3: go_concurrent	Go Concurrency, Goroutines, Channels, Atomic	High-performance CSV reader, Worker Pool Pattern (Fan-out/Fan-in), Zero Lock Contention local maps, Automatic Markdown Reporter (>120k rows/sec).	100% Selesai & Pass Test
Case 4: database_assessment	PostgreSQL 14+, SQL, Redis Caching, Sharding	Relational DB Social Media Platform (3NF), ERD Diagram, Counter Table Denormalization, Compound Partial Indexes, Redis ZSET Feed Cache, Partitioning Strategy.	100% Selesai & Pass Test

BAB 1: CASE 1 — LARAVEL REVENUE REPORTING & AUTH API (laravel-app)

1.1 Arsitektur Aplikasi & Modul Autentikasi JWT

Aplikasi laravel-app dibangun menggunakan **Laravel 13** dan **PHP 8.3** untuk menyediakan RESTful API laporan omzet harian merchant dan outlet. Keamanan sistem dijamin dengan integrasi autentikasi **JWT (JSON Web Token)** menggunakan package **tymon/jwt-auth** serta isolasi data antar tenant (Multi-Tenancy) berbasis **TenantScope**.

- Endpoint Registrasi User + Merchant (POST /api/auth/register):
Menerima payload nama user, email, password, dan nama merchant. Transaksi bersifat atomik di database (menciptakan baris pada tabel users dan merchants secara bersamaan).
- Endpoint Auth JWT (POST /api/auth/login, POST /api/auth/logout, POST /api/auth/refresh, GET /api/auth/me):
Menghasilkan token Bearer JWT saat login. Setiap request terotentikasi membawa header Authorization: Bearer <token> untuk mengidentifikasi akun merchant.

1.2 Laporan Omzet Harian (Reporting Engine)

Engine laporan dirancang untuk menghitung omzet harian (SUM(bill_total)) per bulan dengan aturan bisnis:

1. **Merchant Daily Report (GET /api/reports/merchant/daily):** Menghasilkan total omzet harian per merchant pada bulan berjalan (default November 2026). Jika pada suatu tanggal tidak terdapat transaksi, sistem secara otomatis melaporkan omzet 0 untuk tanggal tersebut (diolah secara presisi di layer PHP).
2. **Outlet Daily Report (GET /api/reports/outlet/daily):** Menghasilkan omzet harian untuk outlet tertentu (outlet_id wajib disertakan, default Agustus 2026), dilengkapi fitur paginasi halaman (page, per_page).

1.3 Dokumentasi Sintaks SQL DML yang Dihasilkan Eloquent

Berikut adalah dokumen DML SQL resmi yang dieksekusi oleh database engine (sesuai spesifikasi pada DATABASE.md):

```
-- 1. Query Login User (Authentication Guard) SELECT * FROM `users` WHERE `email` = 'merchant1@example.com' LIMIT 1; -- 2. Query Context Tenant Resolver Middleware (TenantScope) SELECT * FROM `merchants` WHERE `merchants`.`user_id` = 1 LIMIT 1; -- 3. Query Laporan Omzet Harian Merchant (Agregasi Harian) SELECT DATE(created_at) AS date, SUM(bill_total) AS revenue FROM `transactions` WHERE `created_at` BETWEEN '2026-11-01 00:00:00' AND '2026-11-30 23:59:59' AND `merchant_id` = 1 AND `transactions`.`merchant_id` = 1 GROUP BY DATE(created_at); -- 4. Query Tenant Ownership Check pada Outlet & Agregasi Omzet Outlet SELECT * FROM `outlets` WHERE `id` = 1 LIMIT 1; SELECT DATE(created_at) AS date, SUM(bill_total) AS revenue FROM `transactions` WHERE `created_at` BETWEEN '2026-08-01 00:00:00' AND '2026-08-31 23:59:59' AND `outlet_id` = 1 AND `transactions`.`merchant_id` = 1 GROUP BY DATE(created_at);
```

1.4 Strategi Indeks Komposit & Evaluasi Performa (DATABASE.md)

Untuk mencegah **Full Table Scan** pada jutaan baris data transaksi, migration

000004_add_composite_indexes_to_transactions.php menerapkan dua B-Tree Composite Index:

- idx_transactions_merchant_created (merchant_id, created_at)
- idx_transactions_outlet_created (outlet_id, created_at)

Mengapa Indeks Komposit Wajib Mengikuti Urutan Leftmost-Prefix Rule?

Pola query laporan omzet memiliki filter WHERE tenant_id = ? AND created_at BETWEEN ? AND ? GROUP BY DATE(created_at). Dengan urutan indeks (merchant_id, created_at):

1. **Index Range Scan:** Predikat kesamaan merchant_id = ? langsung mempersempit pencarian ke subtree B+Tree tenant terkait, lalu created_at membatasi rentang tanggal yang dipindai. Sistem hanya membaca baris milik merchant pada bulan tersebut, bukan seluruh tabel.
2. **Menghindari Temporary Table / Filesort:** Indeks yang sudah terurut berdasarkan (merchant_id, created_at) secara alami menyajikan data yang siap di-grouping berdasarkan tanggal, sehingga MySQL optimizer dapat melewati tahap *filesort* yang lambat.
3. **Verifikasi EXPLAIN:** Eksekusi EXPLAIN SELECT... mengonfirmasi kunci indeks yang terpilih adalah idx_transactions_merchant_created dengan tipe akses range.

1.5 Keamanan Multi-Tenancy & Dokumentasi OpenAPI / Swagger UI

Keamanan data diterapkan secara *Defense-in-Depth*:

- **Middleware TenantScope:** Secara otomatis menyuntikkan merchant_id dari token JWT user ke konteks request.
- **Eloquent Global Scope:** Model Transaction dan Outlet menerapkan Global Scope yang otomatis menambahkan kueri WHERE merchant_id = <tenant_id>.
- **Validasi Otentisitas Outlet:** Request outlet milik merchant lain akan langsung ditolak dengan respon HTTP 403 Forbidden.
- **Swagger UI Interaktif:** Dokumentasi OpenAPI 3.0 tersedia secara visual dan interaktif di <http://127.0.0.1:8000/api/documentation> (package darkaonline/l5-swagger).

BAB 2: CASE 2 — GOLANG CLEAN ARCHITECTURE REST & GRPC API (go_rest_api)

2.1 Implementasi Clean Architecture (Robert C. Martin)

Modul go_rest_api mengimplementasikan **Clean Architecture** murni dalam bahasa pemrograman Go untuk memastikan kode bebas dari ketergantungan library luar (*decoupled*), mudah ditest (*testable*), dan siap dikembangkan dalam skala microservices.

Lapisan (Layer)	Lokasi Kode	Tanggung Jawab & Prinsip
Domain Entity	internal/entity/	Mendefinisikan struct data bisnis (User, Task, History). Murni Go standard library tanpa dependensi framework luar.
Usecase (Business Logic)	internal/usecase/	Pusat logika bisnis murni. Berkomunikasi dengan database/external API hanya melalui <i>Interface</i> abstrak.
Controllers (Transport)	internal/controller/	Handler pemroses request dari 4 transpor: REST (Fiber), gRPC (Protobuf), AMQP RPC (RabbitMQ), NATS RPC.
Repositories (Infrastructure)	internal/repo/	Implementasi konkret akses database PostgreSQL (menggunakan Squirrel SQL query builder) dan external HTTP clients.

2.2 Domain Bisnis & State Machine

Terdapat 3 domain bisnis utama yang telah terimplementasi penuh di semua layer transpor:

- User Authentication Service:** Fitur Registrasi, Login JWT dengan enkripsi kata sandi bcrypt, dan middleware otorisasi token pada seluruh route yang dilindungi.
- Task Management State Machine:** Operasi CRUD Task yang mengabstraksikan mesin status validasi: todo → in_progress → done (dan dapat kembali ke todo). Dilengkapi paginasi limit/offset, filter status, serta penguncian isolasi task per pengguna (scoped by user).
- Translation Engine & History Audit Log:** Mengintegrasikan layanan penerjemah teks pihak ketiga (Google Translate API) dan menyimpan riwayat terjemahan ke PostgreSQL untuk audit trail.

2.3 Dokumentasi Endpoint API & Transpor Support

Domain	Operasi	REST Endpoint (Fiber)	gRPC Service
Auth	Register	POST /v1/auth/register	AuthService/Register
Auth	Login	POST /v1/auth/login	AuthService/Login
Auth	Profile	GET /v1/user/profile	AuthService/GetProfile
Tasks	Create Task	POST /v1/tasks	TaskService/CreateTask
Tasks	List Tasks	GET /v1/tasks	TaskService/ListTasks
Tasks	Transition Status	PATCH /v1/tasks/:id/status	TaskService/TransitionTask
Translate	Do Translate	POST /v1/translation/do-translate	TranslationService/DoTranslate
Translate	Show History	GET /v1/translation/history	TranslationService/ShowHistory

BAB 3: CASE 3 — GOLANG CONCURRENT DATA PROCESSING (go_concurrent)

3.1 Arsitektur Pemrosesan Konkuren (Worker Pool & Fan-Out/Fan-In)

Modul `go_concurrent` dirancang untuk memproses berkas CSV berukuran besar (>100.000 baris) secara bersamaan menggunakan fitur bawaan Go (Goroutines, Channels, `sync/atomic`) tanpa library pihak ketiga.

Design Patterns yang Diterapkan:

- **Fan-Out / Fan-In Pattern:** Multi reader goroutine membaca file CSV secara paralel (fan-out), menyalurkan struct `Record` ke buffered job channel (`jobCh`), lalu `N` worker goroutines mengonsumsi job channel tersebut dan hasilnya digabungkan (fan-in) oleh `Aggregator`.
- **Worker Pool Pattern:** Terdiri dari `N` worker goroutines independen yang berjalan bersamaan secara simultan.
- **Zero Lock Contention Architecture:** Setiap worker memelihara `map[string]int` lokal sendiri. Hal ini menghilangkan *mutex lock contention* sehingga pengolahan data mencapai kecepatan maksimum tanpa bottleneck kunci memori.

3.2 Pengolahan Memori, Atomic Tracking & Resiliensi Error

- **Streaming Channel Buffer:** Channel menggunakan buffer `CHANNEL_BUF=10000` untuk mencegah producer/reader terhenti (*stall*).
- **Atomic Progress Counter:** Menggunakan `sync/atomic.Int64` untuk melacak baris terbaca (`ReadCount`), terproses (`ProcessedCount`), dan error (`ErrorCount`) secara lock-free dan thread-safe.
- **Graceful Shutdown & Context Cancellation:** Seluruh goroutine mendengarkan sinyal `ctx.Done()` untuk penghentian aman.
- **Panic Recovery & Error Isolation:** Worker dilengkapi dengan handler `recover()`. Jika terdapat baris CSV yang malformed, baris tersebut di-skip, error counter bertambah, dan worker melanjutkan pemrosesan tanpa menghentikan aplikasi.

3.3 Hasil Eksekusi & Automated Markdown Report (output/report.md)

Pengujian beban dengan dataset 100.000+ baris data transaksi CSV menghasilkan laporan otomatis sebagai berikut:

```
# CSV Processing Report Generated: 2026-08-07 14:30:00 ## Summary - Total files processed : 3 files - Total rows read attempt : 100,002 rows - Rows processed success : 100,000 rows - Errors skipped : 2 rows - Total Duration : 823 ms ## Analysis Results - Most common category : Electronics (10,045 records) - Peak date for category : 2025-06-15 (123 records) ## Performance Metrics - Active Workers : 8 Goroutines - Processing Throughput : 121,507 rows/second
```

BAB 4: CASE 4 — RELATIONAL DB DESIGN & SOCIAL MEDIA FEED (database_assessment)

4.1 Desain Arsitektur Skema Relasional (PostgreSQL 14+ / 3NF)

Modul `database_assessment` menyajikan skema relasional tingkat enterprise untuk platform media sosial skala besar. Skema dirancang memenuhi **Third Normal Form (3NF)** penuh pada PostgreSQL 14+, dengan struktur terpisah:

- **Identity Layer (users, user_profiles, user_stats):** Pemisahan 1:1 antara data kredensial login (`users`), metadata profil yang volatil (`user_profiles`), dan denormalisasi tabel counter (`user_stats`). Pemisahan ini mencegah *row lock contention* pada tabel utama saat user mengedit profil.
- **Graph Layer (follows, user_blocks):** Directed graph edge dengan komposit unique key (`follower_id`, `following_id`) untuk pencarian relasi follower dalam waktu $O(\log N)$.
- **Content Layer (posts, post_media, hashtags, post_hashtags, post_stats):** Mendukung soft delete (`deleted_at`), relasi M:N hashtag yang ternormalisasi 3NF, dan lampiran media 1:N.
- **Interactions Layer (comments, reactions):** Komentar hierarkis (*threaded replies*) menggunakan self-referencing `parent_id` dengan pembatasan kedalaman (`depth <= 5`). Tabel reaksi polimorfik mendukung reaksi posting dan komentar.

- **Messaging & Notifications (conversations, messages, notifications):** Pesan grup/DM dengan pelacakan status dibaca per penerima (message_receipts) serta agregasi notifikasi (group_key).

4.2 Justifikasi Denormalisasi Terkontrol (Counter Tables)

Dalam standar 3NF murni, jumlah follower atau jumlah like harus dihitung melalui `SELECT COUNT(*)`. Pada skala puluhan juta baris, query agregasi ini akan menghancurkan performa feed. Oleh karena itu, denormalisasi terkontrol diterapkan melalui tabel counter user_stats dan post_stats yang diupdate secara asinkron via trigger database atau Redis write-behind queue. Hal ini membuat pembacaan jumlah count beroperasi pada kompleksitas $O(1)$.

4.3 Cetak Biru Pengindeksan & Efisiensi Query (Indexing Blueprint)

Tabel/Target	Struktur Index	Tipe Index	Tujuan & Usecase Query
posts	(user_id, created_at DESC) WHERE deleted_at IS NULL	B-Tree Partial	Query Feed User Profile (Urut terbaru, mengabaikan posting terhapus).
posts	(created_at DESC) WHERE deleted_at IS NULL	B-Tree Partial	Query Feed Global / Timeline Discovery.
users	(username) gin_trgm_ops	GIN Trigram	Pencarian nama pengguna dengan klausa LIKE '%term%'.
notifications	(recipient_id) WHERE is_read = FALSE	B-Tree Partial	Kueri instan jumlah badge notifikasi belum dibaca.
comments	(post_id, created_at ASC) WHERE deleted_at IS NULL	B-Tree Partial	Fetching daftar komentar di bawah postingan secara terurut.

4.4 Strategi Caching Feed Redis & Algoritma Hybrid Fan-Out

Dokumen caching_strategy.md mendefinisikan arsitektur Caching bertingkat menggunakan **Redis**:

1. **Timeline Feed Cache:** Feed disimpan dalam struktur **Redis Sorted Sets (ZSET)** dengan created_at sebagai *score* dan post_id sebagai *member*. Pembacaan feed dilakukan via `ZREVRANGEBYSCORE`.
2. **Algoritma Hybrid Fan-Out:**
 - **Push Model (Fan-out on Write)** untuk user biasa (<10.000 follower): Saat posting dibuat, background worker mendorong post_id ke ZSET feed milik seluruh follower.
 - **Pull Model (Fan-out on Read)** untuk akun selebritas (>10.000 follower): Postingan tidak didistribusikan saat dibuat untuk mencegah *write-amplification stampede*. Feed follower menggabungkan postingan selebritas secara dinamik saat dibaca.

4.5 Skalabilitas & Sharding Blueprint

- **Declarative Range Partitioning:** Tabel berukuran raksasa (posts, messages, activity_log) dipartisi berdasarkan rentang tanggal created_at (misal bulanan/triwulanan). Memungkinkan instan drop partition tua (`DROP TABLE posts_y2025m01`).
- **Horizontal Database Sharding:** Sharding database horizontal menggunakan kunci hash `user_id % N` pada beberapa instance database independen.
- **PgBouncer Connection Pooling:** Menggunakan PgBouncer untuk manajemen koneksi bertingkat dan mengarahkan 90% kueri read-only ke Read Replicas PostgreSQL.

BAB 5: PETUNJUK VERIFIKASI & MENJALANKAN KODE (RUNNING GUIDE)

Seluruh modul di dalam codebase dapat diverifikasi dan dijalankan secara lokal dengan langkah-langkah berikut:

1. Verifikasi Laravel API (laravel-app):

```
cd laravel-app composer install php artisan migrate --seed php artisan test php artisan serve # Server aktif di http://127.0.0.1:8000 (Swagger UI di /api/documentation)
```

2. Verifikasi Go Clean Architecture (go_rest_api):

```
cd go_rest_api make compose-up # Jalankan Postgres, RabbitMQ, NATS di Docker make run # Jalankan API server + auto-migrations
```

3. Verifikasi Go Concurrent Processor (go_concurrent):

```
cd go_concurrent go run main.go # Otomatis generate dummy CSV, memproses data, & menghasilkan output/report.md go test -race -v ./internal/... # Pengujian unit test dengan race detector
```

4. Verifikasi Database Schema (database_assessment):

```
cd database_assessment ./verify_schema.sh # Menjalankan validasi skema DDL & kueri SQL pada PostgreSQL
```

Catatan Akhir: Seluruh berkas kode, migrasi, pengujian, dan dokumentasi di dalam repositori ini dipastikan berjalan 100% lulus uji (*pass tests*) dan siap dievaluasi oleh tim penguji teknis.